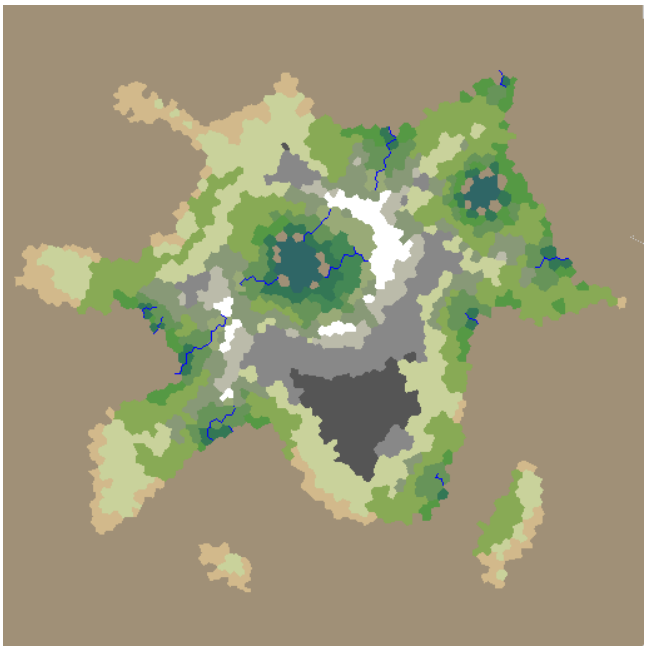
게임 기능에서 제작 도구와 검증까지

Rair / TcDRair · 게임 개발 엔지니어링

C#과 Unity로 게임 기능을 구현하고, 콘텐츠를 만들고 확인하는 도구를 다듬었습니다. 좋아하는 게임의 시스템을 직접 만들어 보는 데서 출발해, 게임 안에서 사용할 수 있는 형태로 연결해 왔습니다.



각 프로젝트에서 수행한 여러 작업 가운데, 구현 과정과 기술적 선택을 되돌아볼 수 있는 사례를 골랐습니다.

사례	주요 내용	쪽
BuilderRPG	랜덤 섬 생성에서 Terrain·프롭 적용까지의 발전 과정과 공간 탐색 개선	2–3
Traveler's Choice · CTS	UI 색상 관리, 상호작용 편집과 어셈블리 참조 설정을 돕는 에디터 도구	4–5
LiC2	역할별 에이전트를 연결한 하네스와 스프린트 1~8의 조율·검증 과정	6–7

주요 기술: C# · Unity · Dictionary · ScriptableObject · EditorWindow

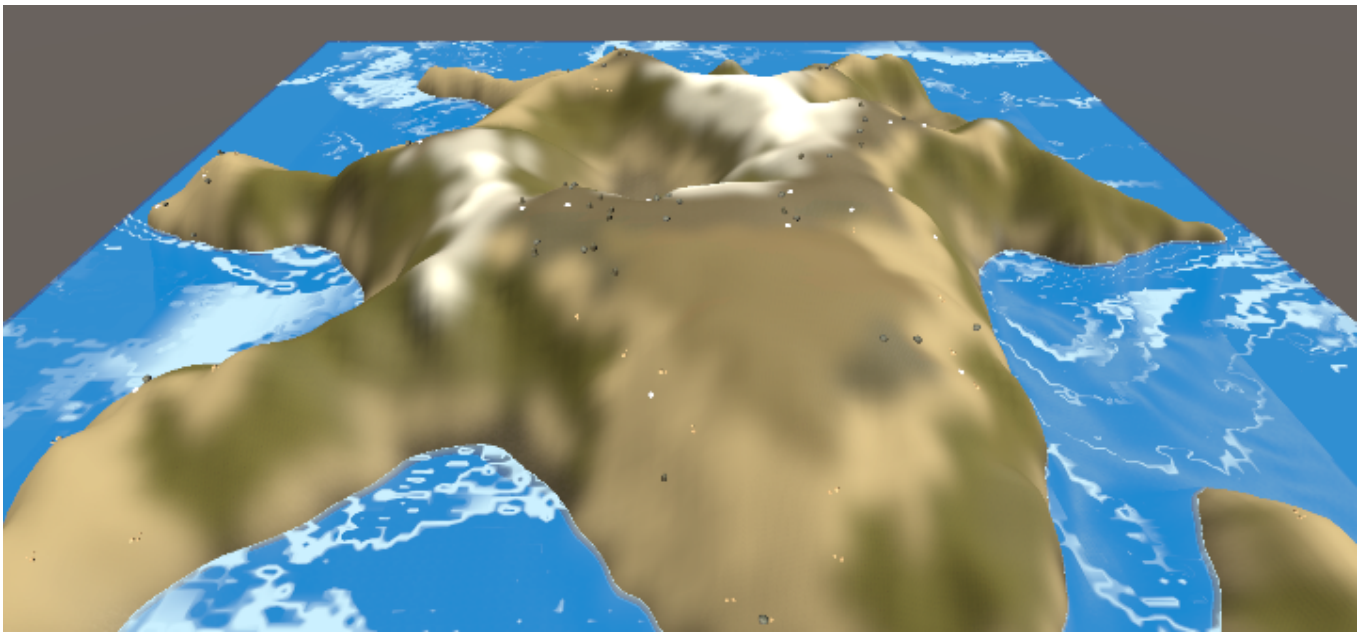
추가 학습: C++ 및 UE5 교육 6개월, Blueprint 프로토타입 개발

GitHub 포트폴리오

랜덤 섬에서 게임 지형으로

BuilderRPG

섬 위를 탐험하고 건설하는 게임을 만들며 외부의 2D 섬 생성기를 Unity Terrain으로 확장했습니다. 지도에 높이·지형 레이어·프롭을 연결하고, 결과를 반복해서 조정할 수 있는 생성 도구를 만들었습니다.



지도 데이터를 높이와 지형 레이어로 변환하고 바이옴에 따라 프롭을 배치한 결과입니다.

지도를 만드는 단계에서 지형을 적용하는 단계로

시기	발전 과정
2022.05	기존 건설 프로토타입에 랜덤 폴리곤 섬 생성기 도입
2022.10	지도와 Height Map을 분리하고 Terrain 변환 실험 시작
2023.06	Terrain 적용과 바이옴별 프롭 자동 배치로 확장
2023.07–08	물 레이어, 부드러운 지형 표현과 URP 전환
2026.07	실행 경로를 정비하고 Terrain 적용·접지 상태까지 검증

고도·습도·바이옴·육지 여부를 RGBA 채널에 저장해 생성 단계와 Terrain 변환기를 연결했습니다. 생성은 코루틴으로 나누고 진행률·취소·재시작을 지원했습니다.

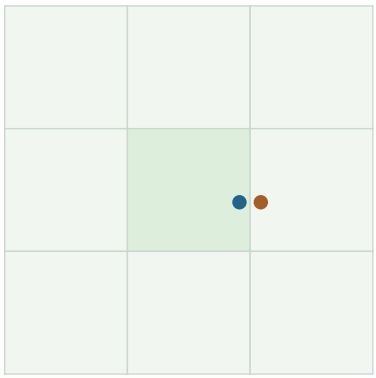
그래프·고도·강·바이옴 알고리즘은 선행 구현을 활용했습니다. 공간 해시, 데이터 전달과 Terrain 변환·생성 도구를 직접 구현했으며, 후기 복구·테스트와 실행 경로 정비에는 AI를 활용했습니다.

탐색과 실행 경로를 개선했습니다

BuilderRPG

그래프를 만들 때 여러 변에서 나타나는 같은 위치의 꼭짓점(Corner)을 하나로 묶어야 했습니다. 기존 List의 x값을 Where로 거르는 방식은 목록 전체를 순회했습니다. Quadtree 등을 검토한 뒤 정수 좌표 셀에 후보를 모으는 Dictionary를 선택했습니다.

같은 셀과 주변 8개 셀에서 가까운 점을 찾습니다



셀 경계의 가까운 두 점도
같은 후보 범위에 포함

```
foreach (var p in box)
  if (vars.cornerMap.TryGetValue(p, out var l)
      && l.Any(c => Vector2.Distance(
          c.point, point) < .01f)) {
    corner = l.First(c => Vector2.Distance(
        c.point, point) < .01f);
    return true;
  }
```

Dictionary<(int, int), List<Corner>>로 셀을 찾고, 원래 좌표 사이의 거리로 같은 점인지 판정했습니다. 전체 목록 대신 주변 셀의 후보를 검사하며, 탐색량은 셀 안의 점 밀도에 따라 달라집니다.

에디터와 런타임의 실행 주체를 분리했습니다

생성 절차를 IEnumerator로 반환하고 실행 주체를 분리했습니다. 에디터에서는 EditorCoroutine으로, 플레이 모드와 빌드에서는 MonoBehaviour의 코루틴으로 같은 절차를 구동합니다.

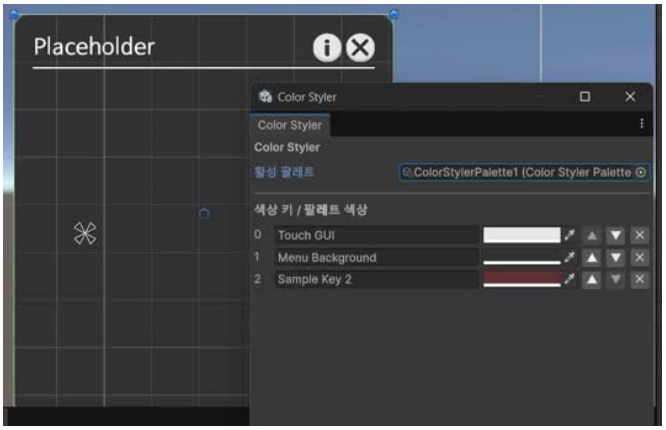
에디터	플레이 모드·빌드
EditorCoroutine으로 실행	StartCoroutine으로 실행
SaveHook으로 PNG 에셋 저장	저장을 건너뛰고 메모리의 텍스처 적용

런타임 호출 경로를 마련했으며 새 게임 시작 UI와의 연결은 남아 있습니다. 탐색 구조 변경 당시의 비교 측정 기록은 남아 있지 않습니다.

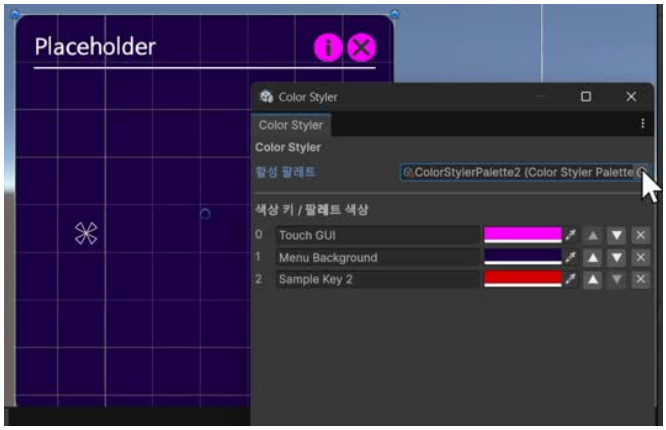
UI 색상을 한곳에서 조정했습니다

Traveler's Choice · Color Styler

여러 UI에서 같은 역할의 색을 유지하면서 다른 색 조합을 빠르게 비교하기 위해 Color Styler를 만들었습니다. 각 Image에 직접 지정하던 색을 색상 키와 팔레트로 나눠 관리했습니다.



팔레트 변경 전



다른 팔레트 적용

팔레트를 바꾸면 배경과 버튼 등 같은 키를 사용하는 UI의 색이 함께 바뀝니다.

색의 역할과 실제 색상값을 분리했습니다

구성	역할
Manager	전역 색상 키와 현재 팔레트 관리
Palette	각 키에 대응하는 Color 값 저장
StyledImage	키에 맞는 색 적용과 변경 알림 구독
EditorWindow	색 조합 편집과 적용 결과 비교

키 이름 변경도 기존 참조에 반영했습니다

열린 씬의 StyledImage와 프리팹 경로를 키별로 캐시했습니다. 프리팹 YAML의 스크립트 GUID로 대상을 좁히고, 키 갱신·Undo 기록·프리팹 저장을 처리했습니다. 닫힌 씬의 일괄 변환은 지원하지 않습니다.

GitHub Copilot의 줄 단위 제안을 활용했습니다. 런타임 팔레트 교체 경로를 마련했으며 플레이어용 테마 선택 메뉴는 미구현입니다.

여러 프로젝트의 에디터 도구

BuilderRPG · Traveler's Choice · CTS

콘텐츠를 편집하고 설정을 반복하는 과정에서 필요한 작업을 인스펙터와 EditorWindow로 옮겼습니다.

BuilderRPG 상호작용 편집

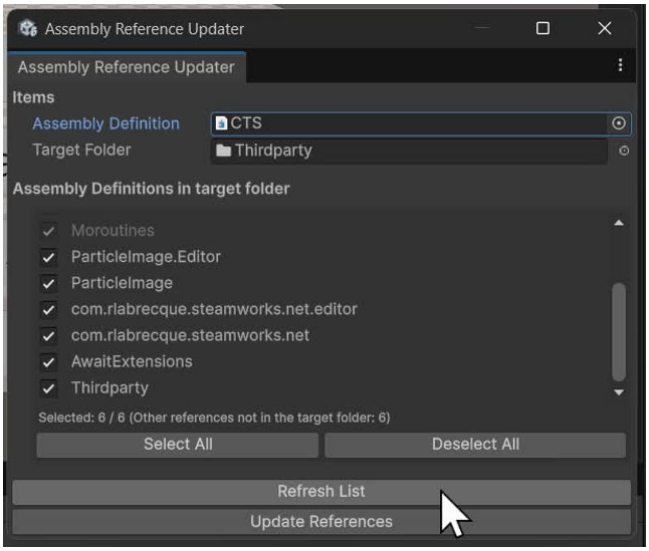
슬롯의 이름·설명·아이콘을 지정하고, 유형에 따라 대기 시간·애니메이션·이벤트를 편집했습니다.



2023년 상호작용 슬롯 인스펙터입니다. 현재는 관련 필드 변경으로 편집 경로가 비활성화되어 있습니다.

CTS 어셈블리 참조 편집

폴더 안의 asmdef를 검색하고 선택한 참조를 대상 어셈블리에 일괄 반영하는 도구입니다.



대상 폴더 밖의 GUID 참조는 유지하고, 선택한 참조를 합쳐 중복을 제거했습니다.

Unity AI Assistant의 도움을 받아 작성했습니다. GUID 기반 참조를 다룹니다.

반복 실행과 참조 확인을 돕는 도구

도구	사용한 곳과 기능
시작 씬 선택기	Traveler's Choice·CTS: 현재 편집 씬과 별도로 Play 시작 씬 지정
Asset ID Viewer	Traveler's Choice: 선택한 에셋의 GUID·Local File Identifier 표시
지형 생성 인스펙터	BuilderRPG: 생성 파라미터와 진행률·취소·재시작 관리

AI 에이전트의 작업을 조율했습니다

LiC2 · 하네스 운영

LiC2는 작업실을 복구하며 탐험하는 1인칭 생존·제작 프로토타입입니다. 역할별 AI 세션을 수동으로 연결하고, 직접 플레이와 코드 확인으로 다음 작업에 필요한 정보를 전달했습니다.

세션 사이에 범위와 완료 기준을 유지하기

LiC에서 AI 기반 프로그래밍을 시도한 뒤, LiC2에서는 여러 시스템을 스프린트 단위로 이어 갔습니다. 기획 의도·구현 범위·완료 기준을 세션 사이에 유지하도록 역할 지침과 스펙·계약·평가 기록, 재검증 절차를 하네스로 구성했습니다.

역할	입력	출력
Planner	설계와 이전 평가·미해결 사항	스프린트 스펙, 범위와 설계 판단
Generator	스펙과 계약 검토 피드백	계약 초안, 구현물, 자체 점검
Evaluator	스펙·계약과 구현물, 플레이 관찰	계약 보완 요청, 평가와 재검증 항목
제 역할	각 세션의 결과와 실제 실행 상태	맥락 전달, 직접 플레이, 코드 확인과 피드백

Generator의 계약을 Evaluator가 검토한 뒤 구현을 진행했습니다. 계약에는 산출물과 함께 조작·기대 결과를 적고, 구현자의 자체 점검과 별도의 평가를 거쳤습니다.

평가 결과에 따라 다음 작업을 연결했습니다

발생한 문제	연결한 작업
구현 결함	Generator 수정 → Evaluator 재검증
계약 밖 요구·설계 모호함	Planner 판단 → 스펙·계약 범위 정리
구조적 결함의 반복	실패 종료와 잔여 문제 기록 → 다음 스프린트 계약

임시로 수정한 코드에서 관찰한 결과와 인도된 구현의 상태도 구분했습니다. 세션이 바뀌어도 무엇을 평가했는지 확인할 수 있도록 결과와 남은 문제를 함께 전달했습니다.

스프린트의 결과를 다음 작업에 연결했습니다

LiC2 · 스프린트와 검증

기반 환경에서 이동·상태·인벤토리·채집으로 구현 범위를 넓혔습니다. 아래는 2026년 계약과 평가 기록에 남은 스프린트별 결과입니다.

회차	작업 범위	조율·검증에서 다룬 점	최종 평가
1	프로젝트·조명·버전 관리	URP의 실제 설정 구조와 문서 의도 대조	PASS 4/26
2	1인칭 조작·상호작용·HUD	입력 방식·폰트 누락 수정 후 재검증	PASS 4/27
3	플레이어 니즈·HUD·효과	로직과 참조 연결, 실제 화면을 함께 확인	PASS 5/7
4	작업실 그레이박스·구획 감지	자동 판정·디버그 오버라이드와 회귀 확인	PASS 5/12
5	디버그 HUD·경계값 검증	검증 도구의 시뮬레이션 덮어쓰기 수정	PASS 5/13
6	인벤토리·아이템 조작	네 차례 평가 후 잔여 결함과 계약 공백 이관	FAIL 5/19
7	UI 상태·인벤토리 재설계	보존 자산과 변경할 상태 모델을 나눠 재검증	PASS 6/10
8	홀드 채집·재생·수율 처리	경계 상태 재현 수단을 보강하고 재검증	PASS 6/24

실패 종료와 계약 재정의

Sprint 6에서는 입력 분기와 아이템 소실 문제가 반복됐습니다. 네 번째 평가에서 FAIL로 종료하고, 남길 자산과 재설계할 문제를 Sprint 7의 입력으로 넘겼습니다. 후속 계약에서 상태 처리와 UI 책임을 다시 정했습니다.

재현 수단까지 다음 세션에 전달하기

Sprint 8에서는 부분 여유 상태를 만드는 디버그 경로가 빠져 있었습니다. 같은 조건을 만들 수 있는 도구를 인도물에 포함하고, 일부 입고·초과 드롭과 배치 충돌을 재검증했습니다.

설계·문서·구현·평가에 AI가 참여했습니다. 저는 세션 조율, 직접 플레이, 코드 확인과 피드백을 맡았습니다. PASS·FAIL은 당시 평가 결과입니다.